

Code Semantic Zooming

Jinsheng Ba
ETH Zurich

Sverrir Thorgeirsson
ETH Zurich

Zhendong Su
ETH Zurich

Abstract—Recent advances in Large Language Models (LLMs) have introduced a new paradigm for software development, where source code is generated from natural language prompts. While this paradigm significantly boosts development productivity, building complex, real-world software systems remains challenging because natural language offers limited control over the code generation process. Inspired by the historical evolution of programming languages toward higher levels of abstraction, we advocate for a high-level abstraction language that gives developers greater control over LLM-assisted code writing. To this end, we propose Code Semantic Zooming (CodeZoom), a novel approach based on pseudocode that allows developers to iteratively explore, understand, and refine code across multiple layers of semantic abstraction. In a within-subjects user study ($n=26$), our method matches a state-of-the-art coding agent, Claude Code, on usability while producing a large effect on code comprehension: over 90% of participants reported feeling more in control of design decisions when using CodeZoom compared to using Claude Code. On objective multiple-choice questions about participants’ own implementations, an exploratory measure added mid-study, a subset of participants ($n=16$) answered 56% of comprehension questions correctly with CodeZoom, compared with 9% with Claude Code (Wilcoxon $p = .005$).

Index Terms—LLM coding, program comprehension, pseudocode

I. INTRODUCTION

In recent years, the paradigm of code writing has undergone a substantial shift driven by Large Language Models (LLMs) [11], [27], [53]. Natural language, most commonly English, has effectively become a new programming language. Developers provide specifications or requirements in natural language prompts, and LLMs generate the corresponding source code. This transformation reduces manual coding effort, allowing developers to focus on higher-level tasks such as design and problem-solving. The approach was termed *vibe coding* [50] and has been rapidly integrated into modern coding tools, including Codex [45], Cursor [47], Gemini CLI [48], Claude Code [44], and Copilot [46].

However, natural language offers insufficient control over the code generation process, making it difficult to develop sophisticated real-world software projects using vibe coding alone. The first challenge lies in intention expression. Unlike traditional programming languages such as Python or C, which provide precise syntax and semantics, natural language is inherently ambiguous, underspecified, and context-dependent. This ambiguity often leads to misinterpretations of requirements, resulting in code that diverges from the developer’s true intent. The second challenge is code comprehension. Although LLMs can generate syntactically correct and seemingly plausible code, there is no guarantee of semantic correctness

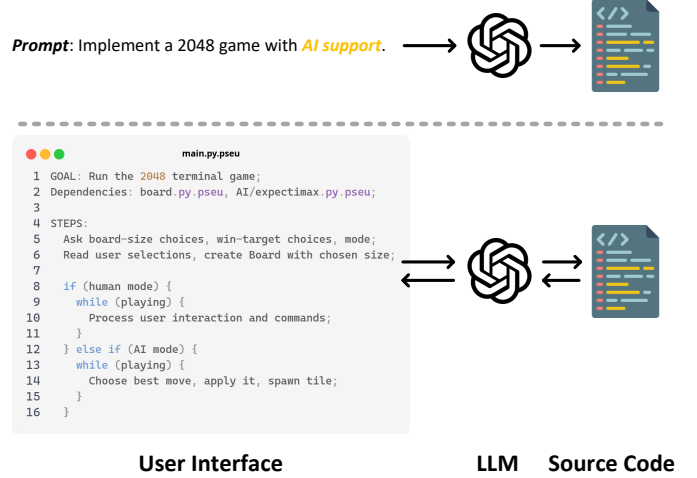


Fig. 1. Existing vibe-coding and CODEZOOM to implement a 2048 game.

or alignment with user intent in natural language. While LLM-generated summaries can provide partial assistance, fully understanding and refining the underlying program logic remains difficult. Developers must still rely on testing or manual inspection to ensure that the generated code satisfies functional requirements. These difficulties are amplified by the iterative nature of software development, where code is continuously revised to add or modify features. Vibe coding offers little control over code generation in such settings. Even if each iteration achieves a correctness rate of 95%, repeated rounds of generation will compound errors, ultimately reducing overall correctness to an unacceptable level.

We envision the need for a high-level abstract language to provide greater controllability in LLM-assisted code writing. Such a language must remain as intuitive as natural language while encoding the structural scaffolding necessary to guide and constrain code generation. While various formalisms could serve this purpose, we instantiate this concept using pseudocode. Because pseudocode is ubiquitous, modern LLMs already possess a robust, pre-trained understanding of its semantics. Pseudocode [35] offers a high-level, human-readable description of program logic that blends natural language with formal programming constructs. Leveraging this dual accessibility, we establish a bidirectional translation mechanism between executable source code and its pseudocode. However, traditional pseudocode is inherently static and cannot capture code semantics at varying levels of granularity. To address this limitation, we propose *Code Semantic Zooming* (CODEZOOM), a grammar-driven approach

that leverages LLMs to dynamically adjust the abstraction level of pseudocode. This approach makes the generated code more controllable, offering both human interpretability and machine guidance.

Figure 1 presents a motivating example of comparing a pure vibe coding tool with CODEZOOM for writing code. While an LLM can generate standard games from pre-trained knowledge alone, the limitations of pure natural language prompts become critical when addressing novel feature requests where pre-trained knowledge falls short.

As shown in the upper half of Figure 1, users write natural-language prompts directly to generate source code. One might provide the prompt “*Implement a 2048 game with AI support*”. However, this prompt can be interpreted in multiple ways, such as whether the AI should fully automate gameplay or merely provide hints to human players. Such distinctions are difficult to express unambiguously in natural language by humans. In contrast, as shown in the lower half of Figure 1, CODEZOOM provides an interface in which users write and edit pseudocode, which LLMs then translate into source code. Operating on pseudocode rather than natural language reduces ambiguity by allowing explicit specification of structural controls such as `if` and `else`. For example, users can clearly indicate that the AI should perform fully automatic gameplay, as a mode distinct from human play. For code validation, CODEZOOM translates the generated source code back into pseudocode, making it explicit that the AI operates as a separate play mode rather than merely providing next-step hints. In addition, the abstraction level of the pseudocode can be adjusted to be more fine- or coarse-grained, enabling users to inspect details at the desired granularity.

We implemented CODEZOOM as a set of skills, which are pre-packaged folders of task-specific instructions for LLM-based coding agents, totaling roughly 2,000 lines, and developed it on top of Claude Code. Because the skill interface is portable, the same implementation can be readily adapted to other agents such as Codex. To evaluate the approach, we conducted a within-subjects user study with 26 graduate students, each of whom completed two programming tasks: implementing a 2048 game from scratch and reproducing a real feature request in an open-source codebase. CODEZOOM was rated as usable as the Claude Code baseline (SUS 69.0 vs. 71.3, $p=0.62$) while delivering substantial gains in code comprehension: on a 7-point Likert item, participants rated their understanding of the codebase at 5.50 with CODEZOOM versus 3.35 with Claude Code ($p<0.001$, Cohen’s $d=1.02$), and on objective multiple-choice questions about their own implementations, participants answered 56% of comprehension questions correctly under CODEZOOM versus 9% under Claude Code (Wilcoxon $p = .005$), and over 90% of participants reported feeling more in control of design decisions when using CODEZOOM rather than Claude Code.

Overall, we make the following contributions:

- We propose a pseudocode approach CODEZOOM for LLM-assisted code writing.

- We introduce Code Semantic Zooming, enabling dynamic control over the granularity of pseudocode.
- A user study ($n = 26$) demonstrates the code-generation controllability gained by CODEZOOM.

II. BACKGROUND

The evolution of programming is defined by a continuous drive toward higher abstraction, aiming to improve expressiveness and maintainability. Early machine and assembly languages offered precise hardware control but were tedious and error-prone. The advent of high-level languages like FORTRAN [5] and COBOL [36] marked a transformative shift, abstracting machine details to democratize software creation and let developers focus on problem-solving.

As software complexity escalated, new paradigms emerged to manage it. Structured programming [12], [15] and C [34] introduced disciplined control flows, while declarative approaches [13], [28] prioritized computational goals over execution steps. Object-oriented programming (OOP) [22], [33], [40] further enabled modular, reusable architectures by encapsulating state and behavior. Today, multi-paradigm languages like Python and Rust [18], [41] dominate, allowing developers to seamlessly blend these styles and, aided by robust package ecosystems [14], drastically accelerate productivity.

Recently, large language models (LLMs) have pushed abstraction further, elevating natural language to a primary interface. Unlike Knuth’s literate programming [24], which intertwined explanatory prose with human-written source code, modern “vibe coding” [50] relies on models to translate plain-language goals directly into execution. This freedom, however, comes at a cost: natural language is inherently ambiguous and underspecified, making it hard to precisely control generated code, validate its correctness, and iteratively refine it for complex systems.

In this work, we argue that the next step in this trajectory should combine natural language with structured, semantically meaningful representations, preserving the expressiveness of natural language while reintroducing the controllability needed for robust LLM-assisted code writing.

III. APPROACH

In this section, we introduce Code Semantic Zooming (CODEZOOM), an approach that enables multiple layers of semantic abstraction of source code using pseudocode, aiming to enhance the controllability of LLM-assisted code writing.

a) Approach overview: Figure 2 presents an overview of CODEZOOM. Suppose we are given a code repository containing four source code files. CODEZOOM adds a pseudocode representation for each source code file and establishes a bidirectional translation between pseudocode and source code, enabling users to inspect and refine source code through pseudocode. In addition, CODEZOOM constructs a navigation file that organizes the overall execution flow across all files. A typical use case begins with developers browsing the navigation file to understand the repository-level execution flow (①). Next, in ②, they navigate to the specific pseudocode file of

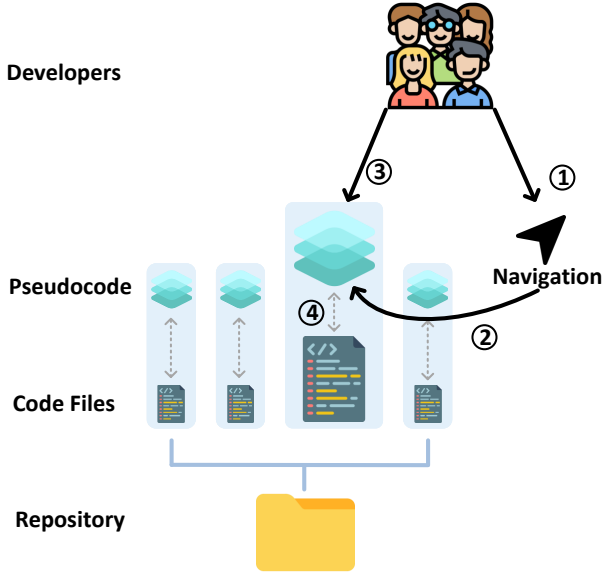


Fig. 2. An overview of CODEZOOM. The system abstracts each code file into a corresponding pseudocode layer. Developers utilize a navigation mechanism to locate specific pseudocode, serving as an interface to interact with and control the corresponding source code.

interest. Finally, in ③, users interact with the pseudocode by dynamically zooming in to inspect detailed logic or zooming out to abstract away low-level details for a higher-level view. This mechanism helps users quickly locate the parts of the code that require modification. Once the desired changes are expressed in pseudocode, they are propagated back to the source code, as shown in ④. The pseudocode is then regenerated from the revised code, allowing users to iteratively verify and refine program semantics until the implementation fully aligns with their intent.

b) Application scenarios: CODEZOOM targets two typical application scenarios across software development.

Scenario 1: Initializing a new code repository. When initiating a new project, developers often start with a high-level design but struggle to maintain control as the implementation grows in scale and complexity, where LLM-generated code frequently drifts away from the intended architecture. With CODEZOOM, developers can first convert their high-level design into coarse-grained pseudocode that captures key logic. They then translate the pseudocode into executable source code and progressively zoom in to refine details. This top-down workflow keeps developers in charge of design decisions while delegating routine implementation to LLMs, thereby enhancing the controllability of LLM-assisted code writing.

Scenario 2: Revising an existing code repository. When developers need to modify an unfamiliar code project, such as adding a feature or fixing a bug in an open-source project from GitHub, they typically face a steep comprehension barrier across hundreds of files. With CODEZOOM, the navigation file offers an immediate high-level view of the execution flow, allowing developers to quickly locate the relevant files. They can then inspect the corresponding pseudocode at a com-

Listing 1. An example of code repository using CODEZOOM.

```
.
|-- README.md
|-- src
    |-- navigation.md
    |-- main.py
    |-- main.py.pseu
    |-- board.py
    |-- board.py.pseu
    |-- AI
        |-- navigation.md
        |-- expectimax.py
        |-- expectimax.py.pseu
```

fortable level of abstraction, zooming in only on the regions that require modification. Edits expressed in pseudocode are propagated back to the source code, sparing developers from wading through unfamiliar low-level syntax while ensuring that changes faithfully reflect their intent.

We explain below how to navigate to specific files and interact with them when using CODEZOOM.

A. Navigation to the Code Files

Listing 1 presents an example code repository using CODEZOOM, which contains the code for a 2048 game. CODEZOOM augments the repository with two categories of auxiliary files, both highlighted in grey in Listing 1: a “navigation.md” file in each subfolder, which provides an overview of all code files in that directory, and a “*.pseu” file, which contains the pseudocode abstraction of each source code file. Users can traverse the “navigation.md” files to locate the specific pseudocode files they are interested in.

Figure 3 shows the content of “navigation.md” under the subfolder “src/” in the example repository from Listing 1. A navigation file consists of two parts: a file list and an execution flow. The first part enumerates all files and subfolders in the current directory, each accompanied by a brief description of its functionality. This enables readers to quickly form a high-level understanding of the role of each file. The second part presents the execution flow across files and subfolders, allowing readers to grasp the overall program structure and locate relevant code. In Figure 3, execution begins at the entry point in “main.py”, where the program first checks whether the user has selected AI mode or human mode to play the game. If AI mode is selected, the program invokes code in the subfolder “AI/” to run the automatic game-playing strategy powered by AI algorithms. In both modes, board-related logic is handled by “board.py”. For example, if a user is interested in the detailed logic of human mode, we could proceed directly to the source-code file “main.py”.

B. Writing Code Files

After locating the target source code file through the navigation files, we open the corresponding pseudocode file and perform inspection and revision. This process involves four stages: converting source code into pseudocode, semantic zooming on the pseudocode, revising the pseudocode, and

src/ — Navigation Map

Files & Subfolders

Item	Purpose
main.py	Entry point — startup menu, human/AI game loops, terminal rendering
board.py	Board class — grid state, moves, tile spawning, undo, win/game-over
AI/	AI package — expectimax search and board-evaluation heuristics

Execution Flow

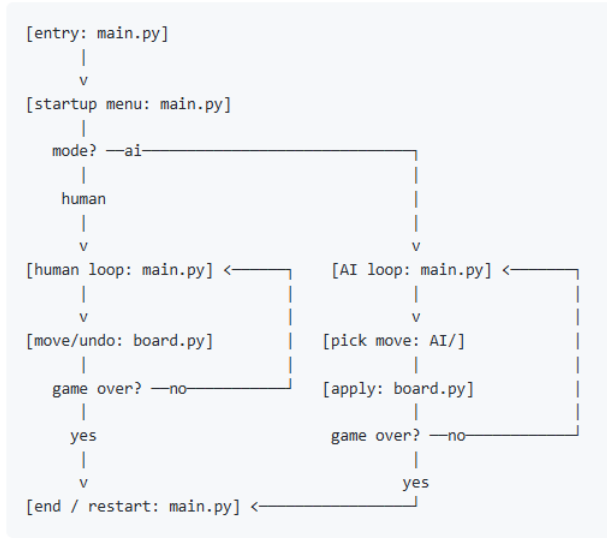


Fig. 3. “src/navigation.md” in Listing 1.

synchronizing changes from the pseudocode back to the source code file. In our example, we open “main.py.pseu”, as shown in Figure 4.

a) *Pseudocode generation*: Each pseudocode representation is generated by CODEZOOM from its corresponding code file. To support semantic zooming over the pseudocode, we define a formal grammar for the representation, as described in Section IV. The translation is performed by LLM-based coding agents, such as Claude Code and OpenAI Codex. Specifically, the prompt provides the pseudocode grammar together with an example. In Figure 4, the generated pseudocode is shown in the top-left corner.

b) *Semantic zooming*: Given the pseudocode, users expand or collapse specific portions to focus on the semantic logic most relevant to their goals if necessary. This step is also done by LLM-based coding agents, allowing users to provide prompts flexibly. In Figure 4, suppose we are curious about what algorithm is implemented for the AI mode, we could zoom in on line 10. Specifically, users

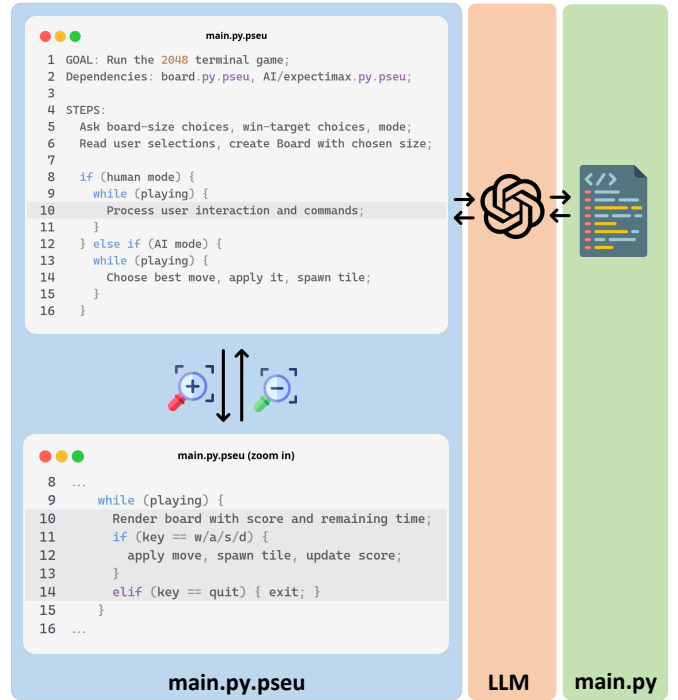


Fig. 4. The bidirectional relationship between “main.py” and its pseudocode “main.py.pseu” from Listing 1. An LLM facilitates seamless conversion between the source code and pseudocode. Developers interact with the system by zooming in/out and directly modifying the pseudocode.

could provide the prompt: “Clarify the AI mode in line 10 at main.py.pseu” to CODEZOOM. Then line 10 in the pseudocode is expanded to lines 10–14 in the bottom-left corner code block. Users could further adjust the abstraction level of the pseudocode until the semantic logic most relevant to their goals is clear enough.

c) *Pseudocode Revision*: Given the adjusted pseudocode, users may revise it to achieve their goals. This step can be performed either manually or by LLM-based coding agents. It is the core user-interaction step in CODEZOOM and the most significant distinction from the user interface of pure LLM-based coding agents. For example, in Figure 4, suppose we would like to add a next move suggestion for human mode, we could manually insert one line “suggest next move based on a simple heuristic algorithm” before line 11 in the “main.py.pseu (zoom in)” in the bottom-left corner of Figure 4. Alternatively, we could instruct an LLM to perform the revision using the prompt: “I want to use a simple heuristic algorithm to suggest next move for human mode in main.py.pseu.”

d) *Pseudocode-code sync*: After the pseudocode is modified, the changes are propagated back to the source code, and the pseudocode is also re-generated from the revised source code. Both steps are done by LLM-based coding agents automatically. To record the changes in the pseudocode, we create a backup file for each pseudocode file. When the pseudocode file is modified, we can compare it to the backup to observe the modified context and pass it to the LLM-

based agents. In Figure 4, CODEZOOM maintains a backup file, ‘.main.py.pseu’, which does not include lines 10–14 of ‘main.py.pseu’. CODEZOOM then compares the differences between the two files and uses them to guide LLMs in revising the corresponding source code file. Afterwards, CODEZOOM regenerates the pseudocode from the revised source code to reflect the latest version of the program. This is particularly useful when LLMs do not correctly update the source code according to the pseudocode changes, as users can further inspect the regenerated pseudocode and perform another round of revision. We argue that CODEZOOM provides an efficient communication channel between humans and LLMs, enabling a controllable and effective refinement process for code writing using LLMs.

C. Implementation

We implemented CODEZOOM as a set of skills. Skills are pre-packaged folders that contain best practices and task-specific instructions for LLM-based agents. Each skill includes a `SKILL.md` file that stores condensed guidance accumulated through iterative experimentation, helping LLMs produce higher-quality outputs for specific tasks. Skills are widely supported by existing LLM-based coding agents, such as Claude Code and Codex. We developed our implementation on top of Claude Code, one of the most popular coding agents. Because the skill interface is portable, we believe CODEZOOM can be adapted by other agent frameworks.

IV. PSEUDOCODE GRAMMAR

A. Motivation

Pseudocode has traditionally been an informal description of program semantics and logic. As a result, there is no universally accepted standard grammar. While this informality benefits human readers, it is insufficient for our setting; the complete lack of grammar introduces architectural ambiguity that LLMs frequently misinterpret during semantic zooming. To resolve this, we adopt a semi-formal approach. We introduce a grammar designed strictly to constrain the program’s control flow (e.g., loops and conditionals), providing the LLM with unambiguous architectural boundaries. Meanwhile, the operational steps within these structures remain unconstrained, informal natural language strings. This ensures the pseudocode remains as easy for humans to read as traditional informal styles, while providing enough structural scaffolding to guide and constrain LLM code generation.

B. Grammar

To formally specify the grammar, we adopt Extended Backus–Naur Form (EBNF) [32], which provides a precise yet concise notation for expressing context-free grammars. Listing 2 presents the resulting pseudocode grammar in EBNF.

The pseudocode consists of three major components: **Goal**, **Dependencies**, and **Steps**. **Goal** is a one-sentence summary that captures the high-level semantics of the source code. **Dependencies** is a list of other code files on which the current file depends. It enables users to navigate to related files when

Listing 2. The grammar of pseudocode in EBNF.

```
Pseudocode ::= Goal Dependencies Steps
Goal ::= 'GOAL:' Description ';'
Dependencies ::= 'Dependencies:' Description ';'
Steps ::= 'STEPS:' (Statement)+
Statement ::= SimpleStmt
                | WhileStmt
                | IfStmt
                | ForStmt
SimpleStmt ::= Description ';'
WhileStmt ::= 'while' '(' Cond ')' '{' (Statement)+ '}'
IfStmt ::= 'if' '(' Cond ')' '{' (Statement)+ '}'
            ( 'elif' '(' Cond ')' '{' (Statement)+ '}'
              ) *
            ( 'else' '{' (Statement)+ '}' ) ?
ForStmt ::= 'for' '(' Cond ')' '{' (Statement)+ '}'
Cond ::= [a-zA-Z0-9]+
Description ::= [a-zA-Z0-9]+
```

the current file does not contain the required semantic logic. **Steps** is an ordered sequence of concrete actions used to realize the goal. Each step corresponds to a **Statement**, which may be either a simple natural-language sentence (i.e., **SimpleStmt**) or a control structure (i.e., **WhileStmt**, **IfStmt**, or **ForStmt**). Each control structure contains a **Cond** and a nested list of **Statement** enclosed in brackets `{ }`. This indicates that when **Cond** evaluates to true, the enclosed statements are executed. Finally, **Description** denotes a natural-language sentence.

Compared to the pseudocode styles proposed for educational purposes [49], our grammar is deliberately simpler: it includes only the minimal set of branch and loop control structures, while all other aspects are expressed in natural language. This design offers several advantages. First, it reduces the learning curve for humans, as the pseudocode closely resembles everyday language and avoids additional syntactic burdens such as explicit variable declarations or rigid typing rules. Second, it provides enough structural scaffolding to help LLMs avoid common pitfalls, such as misinterpreting the intended control flow or introducing spurious logic. Third, the separation between the high-level **Goal** and the procedural **Steps** mirrors how developers naturally think about code: they start with an overall objective and then refine it into smaller, executable actions.

V. EVALUATION SETUP

A. Design

To evaluate our system, we conducted a user study that employed a within-subjects design with two conditions, *Claude Code*, which is a source-code interaction with an LLM assistant, and CODEZOOM, which is our pseudocode-based semantic-zooming workflow, layered on the same LLM assistant. Each participant completed both conditions; order (Claude Code-first versus CODEZOOM-first) was counterbalanced across participants. Each participant solved the same two programming tasks under each condition. The primary confirmatory hypothesis and analysis plan were pre-registered prior to data collection. The report is available here: <https://aspredicted.org/hi97f8.pdf>.

a) *Baseline system*: The control condition used Claude Code v2.1.84 with Claude Sonnet 4.6, which was the latest default model at the time of the study. The CODEZOOM condition was built on top of the same version of Claude Code with the same underlying model, so that the two conditions differ only in the pseudocode interface layer.

b) *Tasks*: Participants completed two programming tasks per condition, selected to cover complementary scenarios described in Section III. To prevent cognitive fatigue from confounding our results [7], [16], we limited the study to two tasks per participant. This approach prioritizes precise behavioral observation over broad task generalization, a standard methodological trade-off in empirical software engineering research [39]. *T1 – Greenfield implementation* asked participants to implement the sliding-tile puzzle 2048 from scratch, matching the behavior of a provided reference binary; T1 primarily exercises design-time control. *T2 – Feature request in an existing codebase* asked participants to implement a feature request for the LLM fine-tuning framework Instruct-Lab (PR #3286)¹: adding default `student_model_id` and `teacher_model_id` fields to the `general` configuration section and propagating them through data generation and training so they take effect unless overridden via the command line. T2 primarily involved code comprehension and targeted modification in an unfamiliar multi-file repository. A recent study [10] showed that T2 is challenging, as all existing LLM-based coding agents cannot successfully solve it.

c) *Procedure*: Sessions were run in person on identical hardware and were proctored by the research team. Due to a limited pool of Claude Code accounts, participants were scheduled in pairs per session. Each session lasted 60 minutes and began with a 5-minute tool-orthogonal tutorial on the condition to which the participant was first assigned. Participants completed the post-condition portion of the survey immediately after each condition, and a final overall-impressions section after both conditions were complete. Each participant received an honorarium equivalent to approximately 30 USD.

B. Participants

32 participants from two local universities signed up for our study, of which $N=26$ attended. The mean age was 24.6 years ($SD = 2.02$; one participant did not report age). Gender distribution was 14 male, 10 female, 1 non-binary, and 1 not reported. The participants were all master’s students in computer science or a related subject. Study year was approximately balanced (1st year: 9; 2nd year: 9; 3rd year or later: 7; 1 not reported). Self-reported programming experience was high (14 reported > 5 years; 10 reported 3–5 years; 1 reported 1–2 years; 1 not reported), and LLM-tool usage was frequent (8 used them daily, 13 weekly, 3 monthly, 1 never, 1 not reported). Order assignment was balanced: 13 participants completed the Claude Code condition first and 13 completed the CODEZOOM condition first.

¹<https://github.com/instructlab/instructlab/pull/3286>

C. Instruments

After each condition, participants completed a survey booklet comprising the System Usability Scale [8]; 10 items scored on a 0–100 scale, where scores above 68 indicate above-average usability and above 80.3 indicate excellent usability; the NASA Task Load Index (NASA-TLX) [19]; six subscales rated 0–100 in steps of 5; a self-report of code understanding (CU, 7-point Likert); and two code-revision items (CR-Locate and CR-Ease, both 7-point Likert). The CODEZOOM booklet also contained a 7-point rating of zoom-command helpfulness (CZ-Zoom) and two 7-point agreement items on whether CODEZOOM improves understanding (CZ-Compare-Understand) and revision (CZ-Compare-Revise) relative to working with source code directly (see Figure 5). To contextualize the responses, students were also invited to answer freeform questions on when CODEZOOM was particularly helpful or unhelpful, and how CODEZOOM could be improved.

D. Protocol Amendment

After the first week of data collection, we amended the survey in two ways, which affected participants 11–26:

- 1) **Addition of an actual-learning measure (RQ2, below)**. The pre-registered confirmatory measure (CU) is a *self-report* of understanding. We realized that behavioral evidence of code awareness would be a useful complement to this subjective measure, because self-reports can dissociate from actual comprehension. We therefore added two task-specific multiple-choice content questions per condition (“QT1” and “QT2”), each with eight answer options, to probe participants’ knowledge of the codebase.
- 2) **Rewording of the agency/control item**. In the original booklet, the CODEZOOM-section item read, “I felt in control of the design decisions when working with CODEZOOM, as opposed to having Claude make all decisions for me” (agree/disagree, 7-point). We realized that this framing is biased and unsatisfactory because it presupposes that Claude “makes all decisions” for the user, which primes agreement with CODEZOOM. We replaced it with a bipolar comparison item administered once after both conditions: “Under which programming mode did you feel more in control of your design decisions?” anchored by 1 = Claude Code and 7 = CODEZOOM.

These changes are deviations from the pre-registered protocol. We report them here in full and treat all analyses that depend on them (RQ2; the bipolar control item) as exploratory and restricted to participants 11–26 ($n=16$). The other pre-registered measures (CU, SUS, NASA-TLX, CR-Locate, CR-Ease, CZ-Zoom, CZ-Compare-Understand, CZ-Compare-Revise) were not affected and use the full sample ($N=26$).

E. Research Questions

Our main research questions were:

RQ1 Do participants using CODEZOOM report higher self-rated code understanding than when using Claude Code alone?

RQ2 Do participants using CODEZOOM show higher actual code awareness, as measured by multiple-choice content questions about the codebase, than when using Claude Code alone?

RQ1 was pre-registered as the confirmatory hypothesis; RQ2 was added as a secondary, pre-specified hypothesis during a protocol amendment after the first week of data collection, with its measure and analysis plan fixed before any of the $n = 16$ amended-protocol data were collected.

We also had exploratory comparisons, which were pre-registered as secondary. We computed paired comparisons of SUS total scores, NASA-TLX and its subscale ratings, CR-Locate, and CR-Ease between conditions. Within the CODEZOOM condition, we further report CZ-Zoom, CZ-Compare-Understand, and CZ-Compare-Revise (each tested against the neutral midpoint of 4), and the bipolar control-preference item (tested against 4, with 1 = Claude Code and 7 = CODEZOOM).

F. Analysis Plan

The confirmatory test for RQ1 was a paired-samples t -test on the code understanding question (CU), with a Wilcoxon signed-rank test substituted if the normality assumption on the difference scores was violated (Shapiro–Wilk, $\alpha = .05$). The same rule was applied to all exploratory paired comparisons. Effect sizes are reported as Cohen’s d for t -tests and the matched-pairs rank-biserial correlation r_{rb} for Wilcoxon tests. For single-sample items, we report one-sample t -tests against the scale midpoint (4) with Wilcoxon as a robustness check. Per the pre-registered exclusion rule, participants who failed to complete both conditions were excluded; none of the analyzed 26 had missing data on the pre-registered primary outcomes. All tests are two-sided; we do not adjust for multiple comparisons given the exploratory framing of the secondary analyses, but readers should interpret secondary p -values with that caveat in mind. Analyses were performed in Python 3.14 using `pandas` and `scipy.stats`.

VI. RESULTS

A. Self-Reported Code Understanding (RQ1)

Participants rated their code understanding substantially higher in the CODEZOOM condition ($M = 5.50$, $SD = 1.10$) than in the Claude Code condition ($M = 3.35$, $SD = 1.79$), a mean difference of 2.15 scale points ($SD_{\text{diff}} = 2.11$; $Mdn_{\text{diff}} = 2$). Twenty of 26 participants rated CODEZOOM strictly higher than Claude Code on this item; 4 rated the two equal and 2 rated Claude Code higher. Shapiro–Wilk on the difference scores was not significant ($W = 0.93$, $p = .081$), so the pre-registered paired-samples t -test was used as the primary analysis. It showed a large and highly reliable effect in favour of CODEZOOM, $t(25) = 5.20$, $p < .001$, $d = 1.02$. A Wilcoxon signed-rank test, reported for robustness, reached

TABLE I
EXPLORATORY PAIRED COMPARISONS, CODEZOOM (CZ) VERSUS CLAUDE CODE (CC) ($N=26$). The p -VALUES WERE NON-SIGNIFICANT (> 0.05).

Measure	CC M	CZ M	p	Effect
SUS	71.3	69.0	0.625	−0.10
TLX: Aggregate	27.7	32.1	0.211	0.25
CR-Locate	4.3	5.0	0.057	0.53
CR-Ease	5.2	5.4	0.480	0.19

the same conclusion, $W = 9.00$, $p < .001$, $r_{rb} = 0.93$. **RQ1 is supported.**

B. Actual Code Awareness (RQ2)

Among the 16 participants (IDs 11–26) who completed the added content questions, correct-response rates were markedly higher in the CODEZOOM condition than in the Claude Code condition on both items. On QT1 (algorithm used by the AI mode), accuracy was 62.5% after CODEZOOM and 6.25% after Claude Code; on QT2 (fallback source of the student model ID), accuracy was 50.0% after CODEZOOM and 12.5% after Claude Code. Aggregated, participants answered 18 of 32 content questions correctly in CODEZOOM versus 3 of 32 in Claude Code. Analyzed as per-participant sums (0–2), the mean score was 1.13 ($SD = 0.81$) for CODEZOOM and 0.19 ($SD = 0.40$) for Claude Code, a mean difference of 0.94 ($SD_{\text{diff}} = 0.93$). Shapiro–Wilk on the difference was significant ($W = 0.87$, $p = .028$), so a Wilcoxon signed-rank test was used as the primary analysis: $W = 4.00$, $p = .005$, $r_{rb} = 0.90$. The paired t -test agreed, $t(15) = 4.04$, $p = .001$, $d = 1.01$.

C. Exploratory: SUS, NASA-TLX, and Code Revision

Table I summarizes the exploratory paired comparisons. There was no reliable difference in SUS total scores (Claude Code $M = 71.25$, CODEZOOM $M = 69.04$; $t(25) = -0.50$, $p = .62$), and the point estimates suggest that participants perceived the two environments to be similarly usable. On the NASA-TLX, the CODEZOOM condition was rated as requiring more mental effort and greater overall effort: Mental Demand, Wilcoxon $W = 89.0$, $p = .047$; Effort, $t(25) = 2.97$, $p = .006$, $d = 0.58$. Physical Demand was also rated higher in CODEZOOM (Wilcoxon $W = 16.5$, $p = .042$, $r_{rb} = 0.64$), though absolute levels on this subscale were low in both conditions ($M_s \leq 17$). Temporal Demand, Performance, and Frustration did not differ. The aggregate NASA-TLX score was numerically higher in CODEZOOM but not statistically different ($M = 32.1$ versus 27.7 , $p = .21$, $d = 0.25$). For the code-revision items, CR-Locate was higher in CODEZOOM ($M = 5.04$) than in Claude Code ($M = 4.27$), but with a difference that was not significant by both the pre-registered Wilcoxon (normality violated, $W_{\text{Shapiro}} = 0.90$, $p = .013$; Wilcoxon $W = 32.0$, $p = .057$) and the t -test ($t(25) = 2.02$, $p = .055$). CR-Ease did not differ between conditions ($M = 5.15$ vs. 5.35 ; Wilcoxon $p = .48$).

ID	Question
<i>Per-condition self-reports</i>	
CU	I was able to understand the overall structure and logic of the codebase with confidence using [CC / CZ].
CR-Locate	I was able to locate the relevant code sections efficiently using [CC / CZ].
CR-Ease	It was easy to make the necessary code changes using [CC / CZ].
<i>CZ-specific items</i>	
CZ-Zoom	I felt the <i>zoom</i> command was helpful for understanding the codebase at different levels of abstraction.
CZ-Compare-Understand	Compared with directly reading source code, CodeZoom makes it easier for developers to understand the codebase.
CZ-Compare-Revise	Compared with manually revising source code, CodeZoom facilitates more efficient localization and modification of relevant code sections.
<i>Agency/control item</i>	
Control (original, P1–10)	I felt in control of the design decisions when working with CodeZoom, as opposed to having Claude make all decisions for me.
Control (revised, P11–26)	Under which programming mode did you feel more in control of your design decisions?
<i>Actual-learning items</i>	
QT1	What algorithm is used in your implementation for the AI mode of 2048?
QT2	Before modification, if the student model ID is not provided via the CLI during data generation, where is the student model retrieved from?

Fig. 5. Custom (non-standardised) survey items used in the study, in addition to the SUS [8] and NASA-TLX [19].

TABLE II
CODEZOOM-SPECIFIC ITEMS, TESTED AGAINST THE NEUTRAL MIDPOINT OF 4 ON A 7-POINT SCALE ($N=26$). THE p COLUMN REPORTS THE PAIRED ONE-SAMPLE t -TEST; WILCOXON SIGNED-RANK GIVES $p < .001$ FOR EACH ITEM AS WELL.

Item	M	SD	p
CZ-Zoom	5.23	1.27	$< .001$
CZ-Compare-Understand	5.65	0.94	$< .001$
CZ-Compare-Revise	5.58	1.03	$< .001$

D. Exploratory: CODEZOOM-Specific Items

All three CODEZOOM-specific items were rated well above the neutral scale midpoint of 4 (Table II). Participants found the zoom command helpful for understanding the codebase at different levels of abstraction ($M = 5.23$, $SD = 1.27$; $t(25) = 4.92$, $p < .001$). They also agreed strongly that, compared with directly reading source code, CODEZOOM makes it easier to understand the codebase ($M = 5.65$, $SD = 0.94$; $t(25) = 9.01$, $p < .001$), and that, compared with manually revising source code, CODEZOOM facilitates more efficient localisation and modification of relevant sections ($M = 5.58$, $SD = 1.03$; $t(25) = 7.83$, $p < .001$).

E. Exploratory: Agency/Control Preference

As noted in Section V-D, the original CZ-Control item is not pooled with the bipolar version because they measure different constructs. For completeness, we report both.

For the de-biased *bipolar* item administered to participants 11–26 ($n=16$; 1 = Claude Code, 4 = neutral, 7 = CODEZOOM), mean preference was 5.38 ($SD = 1.41$, $Mdn = 5$), significantly above the neutral midpoint, $t(15) = 3.91$, $p = .001$; Wilcoxon $W = 14.5$, $p = .005$. A count-based summary is informative: 15 of 16 participants expressed a preference toward CODEZOOM, 0 were neutral, and 1 participant expressed

a preference toward Claude Code (rating = 1). For the *original* CODEZOOM-only biased item administered to participants 1–10, responses were 9×6 and 1×5 ($M = 5.90$, $SD = 0.32$; range 5–6), consistent with the interpretive concern that the item’s framing suppressed disagreement.

F. Exploratory: Task Performance

For completeness, we also analyzed task performance. We include task performance only on the second task (T2), since T1 lacks an automatic oracle for comparing a participant’s from-scratch 2048 implementation against the reference binary, whereas T2 provides ground truth via the corresponding pull report (PR #3286) and the project’s existing test suite. On T2, success rates were higher with CODEZOOM ($M = 0.89$, $SD = 0.32$) than with Claude Code ($M = 0.59$, $SD = 0.50$). Because the paired differences departed significantly from normality (Shapiro-Wilk $W = 0.759$, $p < .001$), we used a Wilcoxon signed-rank test, which confirmed that participants performed reliably better with CODEZOOM than with Claude Code ($W = 13.00$, $z = -2.040$, $p = .024$). A common failure mode of Claude Code was modifying the wrong files. InstructLab’s codebase contains LLM model-specific configuration layers that obscure the location of the InstructLab configuration, making it difficult to localize the files relevant to the task requirements. Additionally, we also observed several instances in which Claude Code attempted to modify test files rather than the corresponding implementation files.

G. Exploratory: Freeform Feedback

The free-form responses were broadly positive about CODEZOOM’s core idea; roughly 21 of 26 respondents expressed a favorable overall view. The clearest consensus forming around the code-to-pseudocode direction as an aid to comprehension (cited by 15 of 26). Participants repeatedly described the

pseudocode as a structural overview that made unfamiliar code faster to grasp; one called it “a clear structure or ‘map’ so you can pinpoint specific information more easily,” another noted that “the pseudocode in front of me was particularly useful throughout as it made comprehension of the same code base super fast,” and several reported that translating code back to pseudocode helped them locate bugs more easily than inspecting model output directly. The most consistent criticism concerned the revision experience (raised by about a third of respondents, or 9 of 26): editing pseudocode by hand felt heavier than conversing with the model, as one participant put it, “changing pseudocode by hand is cumbersome compared to Claude’s fast iteration... I think if zoom also was helpful to edit pseudocode it would benefit a lot.” Relatedly, the most common improvement requests were for tighter prompt-driven editing of pseudocode, automatic synchronization between pseudocode and code, and more process feedback while commands run. A minority were less convinced of the abstraction itself (2–3 respondents); one preferred reading the code directly, and another preferred a specification-based intermediate layer over a pseudocode-based one.

H. Sensitivity Analysis

As our study was within-subjects and is therefore susceptible to learning or priming effects, we ran a sensitivity check on RQ1 and RQ2, using between-subjects comparisons restricted to each participant’s first condition only: the 13 participants who saw Claude Code first contribute their Claude Code responses, the 13 who saw CODEZOOM first contribute their CODEZOOM responses, and no participant contributes to both arms. Both effects survive at $p < .05$. For RQ1 (self-rated code understanding), Welch’s $t(22.8) = 3.36$, $p = .003$, Cohen’s $d = 1.32$ ($M_{\text{ClaudeCode}} = 3.54$, $M_{\text{CODEZOOM}} = 5.69$); the rank-based Mann–Whitney test agrees, $U = 141.5$, $p = .003$. For RQ2 (actual code-awareness score, restricted to participants 11–26 for whom the content questions were administered, $n = 8$ per arm), Welch’s $t(10.3) = 4.02$, $p = .002$, Cohen’s $d = 2.01$ ($M_{\text{ClaudeCode}} = 0.13$, $M_{\text{CODEZOOM}} = 1.25$ out of 2; Mann–Whitney $U = 57.5$, $p = .004$). The point estimates of effect size are larger between-subjects than within-subjects, despite the smaller and unpaired sample. We take this as evidence that the primary findings are not artifacts of within-subject carryover.

VII. DISCUSSION

Interpretation. In our study, we compared CODEZOOM against a programming environment where the participants had unrestricted access to AI support. Our two primary measures, namely self-rated understanding (RQ1) and objective code awareness (RQ2), were strongly in favor of CODEZOOM ($d \approx 1.0$, $p < 0.05$). Furthermore, 15/16 participants preferred CODEZOOM on the de-biased agency item. Against that, SUS was indistinguishable between conditions ($p = .62$), and subscales of the NASA-TLX showed that CODEZOOM required significantly more mental effort (Mental Demand $p = .047$; Effort $d = 0.58$, $p = .006$). Our results lend some

support to the thesis that CODEZOOM can help mitigate risks that are sometimes introduced by AI-augmented programming workflows, namely that developers can lose agency of their work and lack an understanding of their own code. At the same time, this cost did not translate into worse coding outcomes: participants in fact performed materially better with CODEZOOM than in the unrestricted-AI baseline. Our data provides some indication that at least for some programming tasks, CODEZOOM can help the developer recover the grasp of their own code without sacrificing the productivity gains that motivate AI-augmented workflows in the first place.

Zooming Behavior. While subjective ratings indicate that participants found the zoom command helpful ($M = 5.23$), we further analyzed their interaction patterns. Every participant executed at least one zoom-in operation. We observed that participants frequently requested to expand specific lines of pseudocode immediately after generating it from the source code. For example, in T1 (2048), after implementing modifications, participants regenerated the pseudocode and zoomed in on the AI execution block to inspect how the next move was calculated. This enabled them to confirm algorithmic choices (e.g., expectimax) that may otherwise be obfuscated in the initial coarse pseudocode. Conversely, this zoom-out behavior frequently occurred after multiple consecutive zoom-in operations to effectively reset the cognitive context. These behavioral patterns confirm that semantic zooming was utilized not merely as a static reading aid, but as an active, dynamic tool to negotiate the abstraction level required for specific comprehension and modification tasks.

Scope. CODEZOOM is designed to improve the controllability of code generation by providing a new communication interface between humans and LLMs, complementing rather than replacing existing LLM-based coding agents. Our goal is to give developers tighter control over the semantic logic of the generated program so that it faithfully follows their intent. Accordingly, we target experienced programmers who already have a clear picture of how their code should be structured, rather than non-experts who rely on LLMs to propose solutions for coding, such as fixing language-level syntax errors or simple bugs. For such experienced developers, CODEZOOM offers a structured medium in which to specify semantics precisely, enabling LLMs to generate more accurate code with less back-and-forth. We therefore view CODEZOOM as a layer that augments agents such as Claude Code, and we expect practitioners to continue relying on the agent’s native capabilities for issues that lie below the semantic layer CODEZOOM captures, such as build-error diagnostics, test-driven iteration, and runtime debugging.

Necessity. CODEZOOM remains necessary even as LLMs become more capable. CODEZOOM targets *controllability*: how developers communicate with LLM agents, rather than the models’ raw coding ability. These two dimensions are largely orthogonal. Even a model with perfect coding skills cannot infer a developer’s intent beyond what is explicitly communicated. However, natural language is inherently underspecified for program semantics: the same English instruction may cor-

respond to many valid implementations that differ in control flow, data structures, or edge-case behavior. Stronger models may generate more plausible code from vague prompts, but plausibility does not guarantee fidelity to intent, and this gap becomes harder to detect when outputs are consistently fluent. CODEZOOM addresses this challenge by providing developers with a structured medium to express intent precisely and to validate the generated code against it. If anything, more capable models further increase its value: they can follow precise specifications more faithfully, while also scaling the risk of confidently incorrect code from underspecified prompts.

Throughput. The main performance overhead of CODEZOOM is the latency of the LLM round-trip: every zoom, revision, and code–pseudocode synchronization requires at least one call to the underlying agent. Currently, each call passes the relevant source file together with its pseudocode counterpart, which is acceptable for the single-file or few-file programs used in our study. CODEZOOM typical responses complete within a few seconds, but will not scale naively to repositories of thousands of files. Reducing this cost will require smarter prompt construction, such as selectively including only the portions of the repository that the navigation file identifies as relevant to the current edit, and caching pseudocode across calls so that unchanged files are not re-translated.

Threats to validity. Our study has some limitations. The within-subjects design could cause carryover effects, since participants solved the same tasks under both conditions; counterbalancing and the between-subjects sensitivity analysis (Section VI-H) mitigate this to some extent. A single 60-minute lab session with a tool built by the authors could have introduced novelty effects and acquiescence bias. Our study was conducted with master’s students from two universities in the same geographic region. The results may differ for more experienced developers, who may exhibit different work pacing, interruption patterns, or toolchain integration practices. The CODEZOOM system might be less effective for different tasks; we do not claim that it is a general-purpose solution for all software development tasks, but rather that pseudocode-based semantic zooming is a promising design point for tasks centered on code comprehension and targeted modification; we consider our two tasks to be representative instances of that. We also note that both conditions used a single underlying model (Claude Sonnet 4.6) and Claude Code version; the size of CODEZOOM’s benefit may shift with future models.

VIII. RELATED WORK

a) From code to pseudocode: A complementary research direction is translating source code into pseudocode to support code comprehension. Early work in this area primarily relied on rule-based methods. For example, Sridhara *et al.* analyzed function definitions using predefined heuristics to generate summary comments [37], [38], Buse *et al.* designed heuristic rules to summarize execution specifications [9], and Moreno *et al.* focused on summarizing class definitions with similar techniques [30]. While these approaches can produce accurate summaries when their rules align well with the input, they

demand significant manual effort to design and are difficult to generalize across programming languages and projects. More recently, with the advent of LLM, data-driven methods have been proposed to automatically convert source code into pseudocode. Oda *et al.* applied statistical machine translation techniques to this task [31], while Alokla *et al.* fine-tuned BART [26] and proposed a retrieval-augmented transformer for pseudocode generation [2], [3]. Other works demonstrate the utility of pseudocode in educational contexts [23], or directly leverage LLMs for generation [17], [51]. Building on these efforts, CODEZOOM exploits the generative capability of LLMs to directly translate source code into pseudocode, while introducing semantic zooming to improve comprehension.

b) From pseudocode to code: Similarly, recent research has explored diverse approaches for translating pseudocode into executable source code. Search-based methods such as SPoC leverage compiler feedback to iteratively refine candidate programs [25]. Machine learning approaches have also been widely studied: Acharjee *et al.* employ a sequence-to-sequence model for pseudocode-to-code translation [1]. More recently, transformer-based methods have gained traction, with C3PO introducing a lightweight Copy–Generate–Combine mechanism [43], and retrieval-based transformers enhancing translation through external memory [3]. Complementary efforts include survey work highlighting transformer-based translation of pseudocode into Python [42]. Coladder [52] establishes a unidirectional, multi-layer translation from prompts to source code. In this work, we establish a bidirectional translation between pseudocode and executable source code for efficient control of the code.

c) LLM for software engineering: A relevant and broader research direction is leveraging large language models (LLMs) to address problems in software engineering. A recent survey reports that over one hundred papers have already been published in this area [27]. For example, Elicitation [4] and SpecGen [29] employ LLMs to mine user requirements as comprehensively as possible. CodePlan [6] and Agent-Coder [20] adopt multi-agent systems to decompose user requirements into sub-tasks for code generation. Furthermore, SWE-bench [21] has become a widely used benchmark for evaluating LLMs on real-world bug fixing. In contrast to these works, our work focus on the controllability of code writing.

IX. CONCLUSION

In this paper, we introduced *Code Semantic Zooming*, a paradigm for LLM-assisted code writing that uses pseudocode as a zoomable intermediate layer between developers and coding agents. Our approach pairs a deliberately minimal EBNF grammar for pseudocode with a bidirectional translation to source code, and adds a semantic zoom operation that lets developers dynamically adjust the level of abstraction at which they read and edit a program. We realized this concept in CODEZOOM, implemented as a portable set of skills on top of Claude Code, and evaluated it in a within-subjects user study with 26 graduate students on two complementary tasks: a greenfield 2048 implementation and a feature-addition task

in an existing InstructLab codebase. CODEZOOM matched Claude Code on usability while producing a large effect on code comprehension (Cohen's $d = 1.02$; 56% vs. 9% correct on objective content questions) and a clear preference on design control ($>90\%$ preference rate). We argue that this work offers a novel perspective on LLM-assisted coding, paving the way toward a more controllable, comprehensible, and human-in-the-loop paradigm for software development.

REFERENCES

- [1] Uzzal Kumar Acharjee, Minhazul Arefin, Kazi Mojammel Hossen, Mohammed Nasir Uddin, Md. Ashraf Uddin, and Linta Islam. Sequence-to-sequence learning-based conversion of pseudo-code to source code using neural translation approach. *IEEE Access*, 10:26730–26742, 2022.
- [2] Anas Alokla, Walaa Gad, Waleed Nazih, Mustafa Aref, and Abdelbadeeh Salem. Pseudocode generation from source code using the bart model. *Mathematics*, 10(21):3967, 2022.
- [3] Anas Alokla, Walaa Gad, Waleed Nazih, Mustafa Aref, and Abdelbadeeh Salem. Retrieval-based transformer pseudocode generation. *Mathematics*, 10(4):604, 2022.
- [4] Mohammadmehdi Ataei, Hyunmin Cheong, Daniele Grandi, Ye Wang, Nigel Morris, and Alexander Tessier. Elicitron: A large language model agent-based simulation framework for design requirements elicitation. *J. Comput. Inf. Sci. Eng.*, 25(1), 2025.
- [5] John W. Backus. The history of fortran i, ii, and III. In *History of Programming Languages, from the ACM SIGPLAN History of Programming Languages Conference, June 1-3, 1978, Los Angeles, California, USA*, pages 25–74, 1978.
- [6] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D C, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, Balasubramanyan Ashok, and Shashank Shet. Codeplan: Repository-level coding using llms and planning. *Proceedings of the ACM on Software Engineering*, 1(FSE):675–698, 2024.
- [7] Tobias Baum, Kurt Schneider, and Alberto Bacchelli. Associating working memory capacity and code change ordering with code review performance. *Empir. Softw. Eng.*, 24:1762–1798, 2019.
- [8] John Brooke et al. SUS-a quick and dirty usability scale. *Usability evaluation in industry*, 189(194):4–7, 1996.
- [9] Raymond P. L. Buse and Westley Weimer. Automatic documentation inference for exceptions. In *Proceedings of the ACM/SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2008, Seattle, WA, USA, July 20-24, 2008*, pages 273–282, 2008.
- [10] Haorui Chen, Chengze Li, and Jia Li. Featbench: Towards more realistic evaluation of feature-level code generation, 2026.
- [11] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebguss, Alex Nichol, Alex Paino, Nikolai Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *CoRR*, abs/2107.03374, 2021.
- [12] Ole-Johan Dahl, Edsger Wybe Dijkstra, and Charles Antony Richard Hoare. *Structured programming*. Academic Press Ltd., 1972.
- [13] C. J. Date. *A Guide to the SQL Standard, Second Edition*. 1989.
- [14] Alexandre Decan, Tom Mens, and Philippe Grosjean. An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1):381–416, 2019.
- [15] Edsger W. Dijkstra. Letters to the editor: go to statement considered harmful. *Commun. ACM*, 11:147–148, 1968.
- [16] Thomas Fritz, Andrew Begel, Sebastian C. Müller, Serap Yigit-Elliott, and Manuela Züger. Using psycho-physiological measures to assess task difficulty in software development. In *36th International Conference on Software Engineering, ICSE '14, Hyderabad, India - May 31 - June 07, 2014*, pages 402–413, 2014.
- [17] Walaa Gad, Anas Alokla, Waleed Nazih, Mustafa Aref, and Abdelbadeeh Salem. Dibt: Deep learning-based transformer to generate pseudo-code from source code. *Computers, Materials & Continua*, 70(2), 2022.
- [18] Michael Hanus. Multi-paradigm declarative languages. In *Logic Programming, 23rd International Conference, ICLP 2007, Porto, Portugal, September 8-13, 2007, Proceedings*, volume 4670, pages 45–75, 2007.
- [19] Sandra G Hart and Lowell E Staveland. Development of NASA-TLX (task load index): Results of empirical and theoretical research. In *Advances in psychology*, volume 52, pages 139–183. Elsevier, 1988.
- [20] Dong Huang, Qingwen Bu, Jie M. Zhang, Michael Luck, and Heming Cui. Agentcoder: Multi-agent-based code generation with iterative testing and optimisation. *CoRR*, abs/2312.13010, 2023.
- [21] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*, 2024.
- [22] Alan C. Kay. The early history of smalltalk. In *History of Programming Languages Conference (HOPL-II), Preprints, Cambridge, Massachusetts, USA, April 20-23, 1993*, pages 69–95, 1993.
- [23] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Zachary Henley, Paul Denny, Michelle Craig, and Tovi Grossman. Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs. In *Proceedings of the CHI Conference on Human Factors in Computing Systems, CHI 2024, Honolulu, HI, USA, May 11-16, 2024*, pages 650:1–650:20, 2024.
- [24] Donald Ervin Knuth. Literate programming. *The computer journal*, 27(2):97–111, 1984.
- [25] Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy Liang. Spoc: Search-based pseudocode to code. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 11883–11894, 2019.
- [26] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. BART: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 7871–7880, 2020.
- [27] Junwei Liu, Kaixin Wang, Yixuan Chen, Xin Peng, Zhenpeng Chen, Lingming Zhang, and Yiling Lou. Large language model-based agents for software engineering: A survey. *CoRR*, abs/2409.02977, 2024.
- [28] John W. Lloyd. Practical advantages of declarative programming. In *1994 Joint Conference on Declarative Programming, GULP-PRODE'94 Península, Spain, September 19-22, 1994, Volume 1*, pages 18–30, 1994.
- [29] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. Specgen: Automated generation of formal program specifications via large language models. In *47th IEEE/ACM International Conference on Software Engineering, ICSE 2025, Ottawa, ON, Canada, April 26 - May 6, 2025*, pages 16–28, 2025.
- [30] Laura Moreno, Jairo Aponte, Giriprasad Sridhara, Andrian Marcus, Lori L. Pollock, and K. Vijay-Shanker. Automatic generation of natural language summaries for java classes. In *IEEE 21st International Conference on Program Comprehension, ICPC 2013, San Francisco, CA, USA, 20-21 May, 2013*, pages 23–32, 2013.
- [31] Yusuke Oda, Hiroyuki Fudaba, Graham Neubig, Hideaki Hata, Sakriani Sakti, Tomoki Toda, and Satoshi Nakamura. Learning to generate pseudo-code from source code using statistical machine translation (T). In Myra B. Cohen, Lars Grunske, and Michael Whalen, editors, *30th IEEE/ACM International Conference on Automated Software Engineering, ASE 2015, Lincoln, NE, USA, November 9-13, 2015*, pages 574–584. IEEE Computer Society, 2015.
- [32] Richard E. Pattis. Teaching EBNF first in CS 1. In Robert Beck and Don Goelmann, editors, *Proceedings of the 25th SIGCSE Technical Symposium on Computer Science Education, SIGCSE 1994, Phoenix, Arizona, USA, March 10-12, 1994*, pages 300–303. ACM, 1994.
- [33] Tim Rentsch. Object oriented programming. *ACM Sigplan Notices*, 17(9):51–57, 1982.
- [34] Dennis M Ritchie, Stephen C Johnson, ME Lesk, BW Kernighan, et al. The c programming language. *Bell Sys. Tech. J*, 57(6):1991–2019, 1978.

- [35] Geoffrey G. Roy. Designing and explaining programs with a literate pseudocode. *ACM J. Educ. Resour. Comput.*, 6:1, 2006.
- [36] Jean E. Sammet. The early history of COBOL. *ACM SIGPLAN Notices*, 13:121–161, 1978.
- [37] Giriprasad Sridhara, Emily Hill, Divya Muppaneni, Lori L. Pollock, and K. Vijay-Shanker. Towards automatically generating summary comments for java methods. In *ASE 2010, 25th IEEE/ACM International Conference on Automated Software Engineering, Antwerp, Belgium, September 20-24, 2010*, pages 43–52, 2010.
- [38] Giriprasad Sridhara, Lori L. Pollock, and K. Vijay-Shanker. Automatically detecting and describing high level actions within methods. In *Proceedings of the 33rd International Conference on Software Engineering, ICSE 2011, Waikiki, Honolulu , HI, USA, May 21-28, 2011*, pages 101–110, 2011.
- [39] Klaas-Jan Stol and Brian Fitzgerald. Guidelines for conducting software engineering research. In *Contemporary Empirical Methods in Software Engineering*, pages 27–62. 2020.
- [40] Bjarne Stroustrup. An overview of c++. In *Proceedings of the 1986 SIGPLAN workshop on Object-oriented programming*, pages 7–18, 1986.
- [41] Nimit Thaker and Abhilash Shukla. Python as multi paradigm programming language. *International Journal of Computer Applications*, 177(31):38–42, 2020.
- [42] Satya Prakash Tiwari, Shivam Prasad, and MG Thushara. Machine learning for translating pseudocode to python: A comprehensive review. In *2023 7th International Conference on Intelligent Computing and Control Systems (ICICCS)*, pages 274–280. IEEE, 2023.
- [43] Vishruth Veerendranath, Vibha Masti, Prajwal Anagani, and Mamatha HR. C3PO: A lightweight copying mechanism for translating pseudocode to code. In *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing, ACL/IJCNLP 2022 - Student Research Workshop, Online, November 20, 2022*, pages 47–53, 2022.
- [44] Website. Claude code. <https://www.anthropic.com/claude-code>, 2025. Accessed: 2025-09-08.
- [45] Website. Codex. <https://openai.com/codex/>, 2025. Accessed: 2025-09-08.
- [46] Website. Copilot. <https://github.com/features/copilot>, 2025. Accessed: 2025-09-08.
- [47] Website. Cursor. <https://cursor.com/>, 2025. Accessed: 2025-09-08.
- [48] Website. Gemini cli. <https://cloud.google.com/gemini/docs/codeassist/gemini-cli>, 2025. Accessed: 2025-09-08.
- [49] Website. Pseudocode guide for teachers. <https://www.cambridgeinternational.org/Images/697401-2026-pseudocode-guide-for-teachers.pdf>, 2025. Accessed: 2025-09-08.
- [50] Website. Vibe coding. <https://x.com/karpathy/status/1886192184808149383>, 2025. Accessed: 2025-09-08.
- [51] Guang Yang, Yanlin Zhou, Xiang Chen, and Chi Yu. Fine-grained pseudo-code generation method via code feature extraction and transformer. In *28th Asia-Pacific Software Engineering Conference, APSEC 2021, Taipei, Taiwan, December 6-9, 2021*, pages 213–222, 2021.
- [52] Ryan Yen, Jiawen Stefanie Zhu, Sangho Suh, Haijun Xia, and Jian Zhao. Coladder: Manipulating code generation via multi-level blocks. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology, UIST 2024, Pittsburgh, PA, USA, October 13-16, 2024*, pages 11:1–11:20, 2024.
- [53] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Zican Dong, Yupeng Hou, Beichen Zhang, Yingqian Min, Junjie Zhang, Peiyu Liu, et al. A survey of large language models. *Frontiers of Computer Science*, 20(12):2012627, 2026.